DataCrunch Final Round - Crop Price Prediction

“The Freezer Gambit”

Team Name: Xforce

Table of Contents

2. Introduction / Executive Summary

2.1. Problem Statement Recap

The DataCrunch Final Round competition presents the "Legacy of the Market King: The Freezer Gambit" challenge, set in the fictional kingdom of Agrovia. Following the split of Cornelius Greenvale's agricultural empire, his son Magnus aims to leverage data science to optimize crop sales through his "AgroChill" cold storage network. The core task is to build a forecasting system capable of predicting weekly fresh prices for various fruits and vegetables across different economic centers, four weeks (28 days) into the future. These predictions are crucial for Magnus's "Freezer Gambit" strategy – deciding whether to sell produce immediately or store it for potentially higher prices in future weekly markets, thus maximizing revenue and minimizing spoilage. The solution must also include an API interface for retrieving predictions and submitting new data, packaged within a Docker container.

2.2. Solution Overview

This document details a comprehensive solution developed to address the competition's requirements. The system employs a time series forecasting methodology centered around a highly optimized LightGBM (Light Gradient Boosting Machine) regression model. This model is trained on the provided historical price and weather datasets spanning from 2040 to mid-2043.

To enhance predictive accuracy, a robust feature engineering pipeline was implemented, generating features that capture:

- **Temporal Patterns:** Year, month, week of year, day of week, etc.
- **Autocorrelation:** Lagged price values from 4, 5, and 6 weeks prior.
- **Recent Trends & Volatility:** Rolling window statistics (mean, standard deviation) of prices over the past 7, 14, and 28 days.

Model hyperparameters were meticulously tuned using the Optuna framework to minimize the Root Mean Squared Error (RMSE) on a time-based validation set.

The complete solution, including the data processing pipeline, the trained model, and the prediction logic, is exposed via a RESTful API built using the modern FastAPI framework. This API adheres to the specifications outlined in api.yml, providing endpoints for prediction retrieval and placeholder endpoints for data submission. The entire application is containerized using Docker, ensuring reproducibility and ease of deployment, and has been deployed to a public cloud server (Digital Ocean).

2.3. Key Results

The developed forecasting system demonstrates strong predictive performance on the validation dataset (the final two months of the training period).

- **Final Validation RMSE:** 20.06 Silver Drachma/kg. This indicates a high degree of accuracy, with the model's predictions being, on average, very close to the actual prices in the validation period.
- **API Implementation:** All required API endpoints (/api/predict, /api/data/weather, /api/data/prices) have been implemented according to the api.yml specification, with the prediction endpoint fully functional.
- **Containerization & Deployment:** The solution is successfully packaged into a runnable Docker image (mehararothila/data-crunch-predictor:v1.1) available on Docker Hub and deployed on a DigitalOcean Droplet, accessible via IP and domain name, meeting the core and bonus deliverable requirements.

This result positions the solution competitively based on the accuracy evaluation metric, achieved through careful feature engineering and automated hyperparameter optimization.

3. Data Understanding and Preparation

A thorough understanding and careful preparation of the provided data are foundational to building an accurate forecasting model.

3.1. Dataset Description

The solution utilizes two primary datasets provided for the competition, split into training (train_data.csv) and evaluation (eval_data.csv) files:

Weather Data (WeatherData/): This dataset contains historical daily weather records for different regions within Agrovia, covering the period from 2040 to the end of 2043. Key columns relevant to the model include:

- **Date:** The specific date of the weather record (YYYY-MM-DD format).
- **Region:** The market region (e.g., "Mystic Falls", "Valhalla"). Categorical text data.
- **Temperature (K):** Daily average temperature in Kelvin. Numerical data.
- **Rainfall (mm):** Daily total precipitation in millimeters. Numerical data.
- **Humidity (%):** Daily average relative humidity percentage. Numerical data.
- **Crop Yield Impact Score:** A pre-calculated metric quantifying the impact of environmental conditions on crop yield (higher values indicate more favorable conditions). Numerical data.

Price Data (PriceData/): This dataset contains historical daily market prices for various agricultural commodities across the regions, covering the same period. Key columns include:

- **Region:** The market region. Categorical text data, matching the Weather data.
- **Commodity:** The agricultural product being sold (e.g., "Cantaloupe", "Okra"). Categorical text data.
- **Price per Unit (Silver Drachma/kg):** The target variable we aim to predict. Numerical data (float).
- **Type:** Classification as "Fruit" or "Vegetable". Categorical text data.
- **Date:** The specific date of the price record (YYYY-MM-DD format). Used for merging with weather data and for time series analysis.

The training data spans from 2040-01-01 to 2043-06-30. The evaluation data (used implicitly by the judges, snippets provided) covers 2043-07-01 to 2043-12-31.

3.2. Initial Data Analysis (EDA)

Preliminary analysis revealed several key characteristics:

- **Data Granularity:** Both datasets are provided at a daily level.
- **Missing Values:** Initial checks on the training data snippets showed no missing values in the core price or weather columns.
- **Duplicate Weather Entries:** A significant finding was the presence of duplicate weather entries for the same Date and Region. This indicated that the weather data was likely recorded multiple times or structured differently than the price data (where multiple commodities exist for the same date/region). This necessitated a deduplication step during data loading.
- **Data Volume:** While snippets were provided, the full dataset size (implied to be large, potentially millions of rows) necessitates efficient processing, influencing choices like using category dtypes and efficient libraries like LightGBM.

(Note: Further EDA, such as plotting time series for specific crops/regions or analyzing distributions, could provide deeper insights but was balanced against development time.)

3.3. Data Loading (data_loader.py)

The deployment/data_loader.py script handles the initial loading and merging:

- Reads both PriceData/train_data.csv and WeatherData/train_data.csv using pandas.
- Crucially uses the parse_dates=['Date'] argument during loading to ensure the 'Date' column is correctly interpreted as datetime objects.
- **Handles Duplicates:** Before merging, it drops duplicate rows from the weather data based on the combination of Date and Region, keeping only the first occurrence encountered. This prevents artificial inflation of rows during the merge.
- **Merges Data:** Performs a left merge of the price data with the deduplicated weather data, using Date and Region as the keys. This

ensures all price records are kept, and the corresponding unique daily weather information for that region is added.
- Sorts the final merged DataFrame by Date, Region, and Commodity for consistent processing.

3.4. Preprocessing (preprocessing.py)

Following loading, the deployment/preprocessing.py script performs basic cleanup:

- **Column Renaming:** Renames the verbose 'Price per Unit (Silver Drachma/kg)' column to simply 'Price' for easier referencing in subsequent code.
- **Type Conversion:** Converts categorical text columns (Region, Commodity, Type) to Pandas' category data type. This is more memory-efficient than the default object type for columns with repeated string values and can sometimes speed up operations like grouping.
- **Numeric Type Check:** Ensures numeric columns maintain appropriate types (float64).

This prepared DataFrame serves as the input for the feature engineering stage.

4. Feature Engineering (feature_engineering.py)

Feature engineering is a critical step in time series forecasting, transforming raw data into informative features that help the machine learning model capture underlying patterns and dependencies. The deployment/feature_engineering.py script implements the following feature generation strategies:

4.1. Strategy Rationale

The goal was to create features that provide the model with context about:

- **Time:** Seasonality, day-of-week effects, holidays (implicitly via date components).
- **Past Values (Autocorrelation):** How the price on a given day relates to prices on previous days/weeks.
- **Recent Dynamics:** Trends, volatility, and average price levels over recent periods.

4.2. Time-Based Features

These features are extracted directly from the Date column to help the model understand calendar-based patterns:

- **year:** The year of the record.
- **month:** The month of the year (1-12).
- **day:** The day of the month (1-31).
- **day_of_week:** The day of the week (0=Monday, 6=Sunday).
- **day_of_year:** The day number within the year (1-366).
- **week_of_year:** The ISO week number within the year (1-53).
- **is_weekend:** A binary flag (1 if Saturday or Sunday, 0 otherwise).

4.3. Lag Features (Price)

Lag features provide the model with direct information about past price values for the specific time series (Crop/Region combination). This helps capture autoregressive patterns (where past values influence future values).

Implementation: Calculated using pandas.DataFrame.shift() after grouping by Region and Commodity and sorting by Date.

Lags Chosen:

- **Price_lag_28:** Price exactly 4 weeks ago.
- **Price_lag_35:** Price exactly 5 weeks ago.
- **Price_lag_42:** Price exactly 6 weeks ago.

Rationale: These specific lags were chosen because the prediction target is 4 weeks ahead, making prices from 4, 5, and 6 weeks prior potentially strong predictors.

4.4. Rolling Window Features (Price)

Rolling window features summarize the behavior of the price over recent time periods, capturing trends and volatility that might not be captured by single lag points.

Implementation: Calculated using pandas.Series.rolling().agg() after grouping by Region and Commodity, sorting by Date, and applying .shift(1) to the grouped series.

.shift(1) Importance: Applying .shift(1) before calculating the rolling statistic is crucial to prevent data leakage. It ensures that the rolling window feature for a given day only uses data from prior days, simulating the information available at the time of prediction.

Windows Chosen: 7 days (1 week), 14 days (2 weeks), 28 days (4 weeks).

Statistics Calculated:

- **mean:** Average price over the window (e.g., Price_roll_mean_7d). Captures the recent price level/trend.
- **std:** Standard deviation of price over the window (e.g., Price_roll_std_14d). Captures recent price volatility.
- **min_periods:** Set to half the window size to allow calculation even near the beginning of a time series.

(Note: Rolling features for weather variables were initially tested but did not improve validation performance with the current model parameters and were therefore excluded from the final feature set.)

4.5. Handling Missing Values from Feature Generation

Lag and rolling window features inherently produce NaN (Not a Number) values at the beginning of each time series (each Region/Commodity group) where insufficient historical data exists for the calculation.

- **Training:** Rows containing any NaN values are dropped using pandas.DataFrame.dropna() within the model_trainer.py script before splitting the data for training and validation.
- **Prediction:** During prediction (predictor.py), potential NaNs generated for the first few future dates (due to lookback requirements) are handled by forward-filling (ffill), back-filling (bfill), and finally filling any remaining with zero (fillna(0)) to ensure the model receives complete feature inputs.

The resulting DataFrame, containing the original preprocessed data plus these engineered features (and with initial NaN rows removed/handled), forms the complete input dataset for the model.

5. Model Development (model_trainer.py)

This section details the process of selecting, training, tuning, and evaluating the forecasting model.

5.1. Model Selection

The LightGBM (Light Gradient Boosting Machine) Regressor (lightgbm.LGBMRegressor) was chosen as the primary forecasting model. The rationale for this choice includes:

- **High Performance:** LightGBM is well-regarded for its state-of-the-art performance on tabular datasets, often achieving high accuracy.
- **Speed and Efficiency:** It employs techniques like gradient-based one-side sampling (GOSS) and exclusive feature bundling (EFB) to train significantly faster and use less memory than traditional gradient boosting methods like XGBoost, which is crucial given the potential dataset size and resource constraints (<2GB RAM).
- **Categorical Feature Handling:** LightGBM can handle categorical features directly (when specified via the categorical_feature parameter), eliminating the need for explicit one-hot or target encoding in many cases, simplifying the feature engineering pipeline.
- **Regularization:** Includes built-in L1 and L2 regularization parameters (reg_alpha, reg_lambda) to help prevent overfitting.

5.2. Validation Strategy

A robust validation strategy is essential for time series forecasting to accurately estimate how the model will perform on future, unseen data.

Method: A time-based split was implemented. The dataset (after feature engineering and NaN removal) was sorted chronologically by Date.

Split Point: The data was split based on a cutoff date calculated as the maximum date in the dataset minus two months (VALIDATION_MONTHS = 2).

- **Training Set:** All data points up to and including 2043-04-30.
- **Validation Set:** All data points after 2043-04-30 (i.e., May and June 2043).

Rationale: This approach simulates a real-world scenario where the model is trained on past data and tested on its ability to predict the immediate future. It avoids data leakage common with random cross-validation in time series, where the model might inadvertently learn from future data points during training.

5.3. Hyperparameter Tuning (Optuna)

To optimize the LightGBM model's performance beyond default settings, the Optuna framework was employed for automated hyperparameter tuning.

Objective: The optimization goal was set to minimize the Root Mean Squared Error (RMSE) calculated on the time-based validation set defined above.

Search Space: Optuna explored a defined range of key LightGBM hyperparameters, including:

- learning_rate
- num_leaves (controls tree complexity)
- max_depth (limits tree depth)
- reg_alpha (L1 regularization)
- reg_lambda (L2 regularization)
- colsample_bytree (feature fraction per tree)
- subsample (data fraction per iteration - bagging)
- min_child_samples (minimum data required in a leaf node)

Process: Optuna ran 30 trials. In each trial, it selected a combination of hyperparameters, trained a LightGBM model using these parameters (with early stopping based on validation RMSE), and recorded the resulting validation RMSE. Optuna uses intelligent search algorithms (like TPE) to efficiently explore the hyperparameter space and find combinations that yield lower RMSE values.

Best Parameters Found: The study identified the following hyperparameters as yielding the best validation RMSE (20.06):

Note: These parameters correspond to Trial 23 in the Optuna output

{

'learning_rate': 0.09661418124610441,

'num_leaves': 57,

'max_depth': 9,

'reg_alpha': 0.02228897246803973,

'reg_lambda': 0.28168407023100606,

'colsample_bytree': 0.8603855854493329,

'subsample': 0.6641597719662065,

'subsample_freq': 3,

'min_child_samples': 36

}

5.4. Final Model Training

After identifying the optimal hyperparameters via Optuna, a final LightGBM model was trained using:

- The entire training portion of the time-split data (up to 2043-04-30).
- The best hyperparameters found by the Optuna study (from Trial 23).
- An increased number of potential estimators (n_estimators=1500) combined with early stopping (stopping_rounds=100) based on the validation set performance. This allows the model to train until performance on the validation set stops improving, preventing overfitting while potentially using more boosting rounds than explored during the faster Optuna trials.

The resulting trained model object was serialized and saved to deployment/lgbm_price_model.joblib using joblib. The list of features used for training was also saved to deployment/model_features.joblib.

5.5. Performance Evaluation

The primary metric used for evaluation, aligning with the competition criteria, is the Root Mean Squared Error (RMSE).

Calculation: RMSE = sqrt(mean((actual_price - predicted_price)^2))

Interpretation: It measures the standard deviation of the prediction errors (residuals), providing a measure of the average magnitude of the error in the same units as the target variable (Silver Drachma/kg). A lower RMSE indicates better model accuracy.

Final Validation RMSE: The final model, trained with the optimized hyperparameters, achieved an RMSE of 20.06 on the time-based validation set (May-June 2043). This extremely low value signifies that the model's predictions were, on average, only about 20.06 Silver Drachma/kg away from the actual prices during the validation period, indicating a very high level of accuracy for this dataset and feature set.

5.6. Feature Importance

LightGBM provides a mechanism to estimate the importance of each feature in the trained model (typically based on the number of times a feature is used in splits or the total gain brought by splits involving that feature). Analyzing feature importance helps understand which factors drive the model's predictions.

Top 15 Features (Final Tuned Model):

1. Commodity (Most important)
2. Region
3. Price_lag_42
4. Price_lag_35
5. Price_lag_28
6. Price_roll_mean_7d
7. Price_roll_std_7d
8. Price_roll_std_14d
9. Price_roll_mean_14d
10. Price_roll_std_28d
11. Price_roll_mean_28d
12. Rainfall_mm
13. CropYieldImpactScore
14. Humidity_pct
15. Temperature_K

Insights: This ranking confirms the high predictive power of the specific Commodity and Region. Crucially, it also highlights the significant contribution of the engineered features: past prices (lag features) and recent price dynamics (rolling features for both mean and standard deviation/volatility) are ranked very highly, often above the raw weather metrics. This validates the effectiveness of the chosen feature engineering strategy. (Note: Feature importance rankings slightly shifted compared to the non-tuned model, but the overall pattern remains similar).

6. System Architecture and Implementation

This section describes the overall structure of the solution and the roles of its key components.

6.1. Overview

The solution follows a modular design, separating concerns into distinct Python scripts organized within the deployment package. The overall workflow for training and prediction is as follows:

Training Workflow:

1. Raw CSV data (Price, Weather) is loaded and merged (data_loader.py).
2. Basic preprocessing is applied (preprocessing.py).
3. Informative features (time, lag, rolling) are engineered (feature_engineering.py).
4. The data is split chronologically, hyperparameters are tuned using Optuna, and the final LightGBM model is trained and evaluated (model_trainer.py).
5. The trained model (lgbm_price_model.joblib) and the list of features used (model_features.joblib) are saved to the deployment directory.

Prediction Workflow (within API):

1. The FastAPI application (main.py) starts, loading the saved model, feature list, and historical data context using a startup_event.
2. A POST request containing a crop and region arrives at the /api/predict endpoint.
3. main.py determines the future dates (next 28 days) and filters the loaded historical data for the requested crop/region.
4. main.py calls the predictor.generate_future_features() method, passing the filtered historical data and future dates.
5. predictor.py uses the feature_engineering.py functions to generate the required features (time, lag, rolling) for the future dates, leveraging the provided historical context and forward-filling missing weather/generated feature values.
6. predictor.py returns the complete feature DataFrame for the future dates to main.py.

7. main.py calls predictor.predict() with the generated features.
8. predictor.py uses the loaded LightGBM model to predict prices for the future dates.
9. main.py receives the predictions, formats them into the specified JSON response structure, and returns the response to the client.

This entire system is packaged within a Docker container for deployment.

6.2. Module Descriptions (Final)

- **deployment/data_loader.py:** Handles loading, merging, date parsing, and initial cleaning (duplicate removal) of the input CSV datasets.
- **deployment/preprocessing.py:** Performs basic data transformations like column renaming and efficient type conversion (e.g., to category).
- **deployment/feature_engineering.py:** Generates time-based, price lag, and price rolling window features essential for the model's performance.
- **deployment/model_trainer.py:** Orchestrates the model training process, including data splitting, Optuna hyperparameter tuning, final model fitting, evaluation (RMSE calculation), and saving the model artifacts (.joblib files). Includes logic to print full feature importance.
- **deployment/predictor.py:** Contains the Predictor class responsible for loading the saved model and feature list. Includes logic (generate_future_features) to create the necessary features for new prediction requests, handling future weather data via forward-filling and filling initial NaNs in generated lag/rolling features using ffill/bfill/fillna(0). Also contains the predict method to apply the loaded model.
- **main.py:** The FastAPI application server. Defines API endpoints, uses a startup_event to load the predictor and historical data context, handles incoming requests, interacts with the Predictor class, formats API responses, and includes a redirect from the root URL (/) to the documentation (/docs).
- **requirements.txt:** Lists all necessary Python packages and versions.
- **Dockerfile:** Defines the steps to build the container image, including installing system dependencies (like libgomp1 needed by LightGBM), Python packages, copying code/data, and setting the run command.

6.3. API Implementation (main.py) (Final)

Framework: FastAPI was chosen for its high performance, asynchronous capabilities, automatic data validation using Pydantic models, and automatic generation of interactive API documentation (Swagger UI).

Initialization: Uses a FastAPI startup_event to load the Predictor (which loads the model and feature list) and the necessary historical data context into memory once when the application starts, ensuring readiness for prediction requests.

Endpoints:

- **/ (GET):** Redirects users automatically to /docs for convenience.
- **/api/predict (POST):** Implements the core prediction logic as described in the workflow above. Uses Pydantic models (PredictionRequest, PredictionResponse) for request validation and response formatting.
- **/api/data/weather (POST):** Accepts weather data submissions according to the api.yml schema. Currently logs receipt but does not store or process the data.
- **/api/data/prices (POST):** Accepts price data submissions according to the api.yml schema. Currently logs receipt but does not store or process the data.

Server: Uvicorn is used as the ASGI server to run the FastAPI application.

6.4. Prediction Logic (predictor.py) (Final)

The Predictor class encapsulates the prediction-time operations:

Initialization: Loads the lgbm_price_model.joblib and model_features.joblib files during application startup (via main.py).

Feature Generation: The generate_future_features method is crucial. It takes recent historical data for a specific crop/region and the future dates. It forward-fills weather data from the last known historical point. It then re-applies the exact same feature engineering steps (time, lag, rolling) used during training. Crucially, it then forward-fills (and back-fills/fills with zero if necessary) any remaining NaNs in the generated lag and rolling features for the future dates to handle edge cases at the start of the prediction period. This ensures consistency and completeness of features fed to the model.

Prediction: The predict method takes the fully generated and cleaned future features, ensures they match the expected features list, and uses the loaded LightGBM model's .predict() method to generate the price forecasts.

6.5. Containerization (Dockerfile)

The Dockerfile provides a reproducible environment:

- **Base Image:** Uses python:3.13-slim for a relatively small base.
- **System Dependencies:** Installs libgomp1 using apt-get, which is required for LightGBM to run correctly in the minimal Linux environment.
- **Python Dependencies:** Installs packages from requirements.txt using pip.
- **Code & Data:** Copies the application code (main.py, deployment/ folder) and necessary historical training data (Datasets/) into the image. Note: Copying raw data increases image size; alternative strategies exist for production but this ensures functionality for the competition.
- **Execution:** Sets the default command (CMD) to run the Uvicorn server, exposing the FastAPI application on port 8000.

7. API Specification

The API implemented in main.py adheres to the OpenAPI specification provided in the api.yml file.

7.1. OpenAPI Specification Reference

The detailed structure of requests and responses, including data types and examples, is formally defined in the api.yml file provided with the competition materials. The FastAPI application uses Pydantic models derived from this specification for validation and serialization.

7.2. Implemented Endpoints

The following endpoints are available:

POST /api/predict

- **Summary:** Predict future prices for a given crop and region.
- **Request Body:** Requires a JSON object containing crop (string) and region (string).

```
{
  "crop": "Cantaloupe",
  "region": "Valhalla"
}
```

- **Response Body (200 OK):** Returns a JSON object containing the input crop and region, and a list named predictions. Each item in the list represents a daily prediction for the next 28 days and includes prediction_index (0-27), date (YYYY-MM-DD), and predicted price (float, rounded to 2 decimals).

{

"crop": "Cantaloupe",

"region": "Valhalla",

"predictions": [

{ "prediction_index": 0, "date": "2043-07-01", "price": 194.28 }, // Example price

{ "prediction_index": 1, "date": "2043-07-02", "price": 198.44 }, // Example price

// ... up to index 27

]

}

- **Error Responses:** Can return 422 Unprocessable Entity for invalid request format, 500 Internal Server Error for prediction/feature generation failures, or 503 Service Unavailable if the model failed to load.

POST /api/data/weather

- **Summary:** Submit new weather data.
- **Request Body:** Requires a JSON object as defined in api.yml (containing date, region, and nested weatherData with optional rainfall, humidity, temp).
- **Response Body (200 OK):**

{ "message": "Weather data received successfully (placeholder implementation)." }

- **Current Implementation:** This endpoint currently acts as a placeholder. It validates the input format against the schema but only logs the received data to the console; it does not store the data or use it for retraining/predictions.

POST /api/data/prices

- **Summary:** Submit new price data.
- **Request Body:** Requires a JSON object as defined in api.yml (containing date, crop, region, and nested priceData with price).
- **Response Body (200 OK):**

{ "message": "Price data received successfully (placeholder implementation)." }

- **Current Implementation:** Similar to the weather endpoint, this is a placeholder that validates input and logs receipt without further processing.

7.3. Interactive Documentation (Swagger UI)

FastAPI automatically generates interactive API documentation using Swagger UI. While the Docker container is running, this documentation can be accessed in a web browser by navigating to the base URL (which redirects to /docs):

- Using IP: http://64.227.137.70:8000/
- Using Domain: http://api.mehara.io:8000/

This interface allows users to:

- View detailed information about each endpoint, including parameters, request bodies, and response schemas.
- Execute API requests directly from the browser for testing purposes.

(Note: The connection uses HTTP and will show a "Not Secure" warning in browsers. This is expected as SSL/HTTPS was not implemented due to time constraints and not being a strict requirement.)

8. Usage Instructions

This section provides instructions on how to build the Docker image and run the application container locally or access the deployed version.

8.1. Prerequisites

- **Docker:** Ensure Docker Desktop (for Windows/Mac) or Docker Engine (for Linux) is installed and running on your system.

8.2. Running the Deployed Version (Recommended for Evaluation)

The application has been deployed to a DigitalOcean Droplet and is publicly accessible.

- **Access API Docs:** Open a web browser and navigate to:
  - http://api.mehara.io:8000/
  - http://64.227.137.70:8000/

    (You will be automatically redirected to the /docs page)
- **Interact:** Use the Swagger UI interface to test the API endpoints as described in Section 7.3.

8.3. Building and Running Locally (Optional)

If you wish to build and run the image locally:

1. **Get Code:** Clone or download the project files from GitHub: https://github.com/mehara-rothila/Data_Crunch-Xforce.git
2. **Navigate:** Open a terminal in the project root directory.
3. **Build Image:** docker build -t data-crunch-predictor .
4. **Run Container:** docker run -p 8000:8000 --rm --name predictor-app data-crunch-predictor
5. **Access Locally:** Open http://localhost:8000/ in your browser.
6. **Stop Container:** Press CTRL+C in the terminal where docker run is executing.

9. Business Insights and Recommendations

Beyond building a predictive model, the project yields actionable insights relevant to Magnus Greenvale's business and the "Freezer Gambit" strategy.

9.1. Key Drivers of Price Variation

Analysis of the model's feature importances reveals the primary factors influencing price predictions:

- **Commodity & Region Specificity:** The most significant drivers are the specific Commodity being sold and the Region where it's sold. This highlights inherent differences in supply, demand, and pricing dynamics across different crops and locations. Strategies likely need to be tailored accordingly.
- **Price History Matters:** Past prices, particularly from 4-6 weeks ago (lag features), and recent price trends/volatility (rolling features) are highly influential. This confirms that prices exhibit strong autocorrelation and momentum/mean-reversion tendencies that the model effectively captures.
- **Weather Influence:** While less dominant than commodity/region or recent price action, weather variables (Rainfall, Humidity, Temperature, Yield Score) still contribute meaningfully to the predictions, indicating their impact on supply or quality perception.

9.2. Application to the Freezer Gambit

The 4-week price forecasts generated by the /api/predict endpoint directly support the core decision of the Freezer Gambit:

- **Sell vs. Freeze Decision:** By comparing the current market price with the predicted prices for the upcoming 1-4 weeks, Magnus can make data-driven decisions. If predicted prices (minus storage and depreciation costs) are significantly higher than the current price, freezing might be optimal. Conversely, if predictions indicate stable or falling prices, selling fresh immediately is likely preferable.
- **Targeted Strategy:** The predictions are specific to each crop and region, allowing for a granular strategy rather than a one-size-fits-all approach.

9.3. Recommendations for AgroChill

Based on the model and the problem context, the following recommendations can be made:

- **Integrate Cost Data:** To fully operationalize the Freezer Gambit, the prediction system should be integrated with data on freezing costs, storage costs, and estimated depreciation rates for each commodity. This would allow for calculating a predicted net profit for freezing vs. selling fresh.
- **Develop Decision Support Tool:** Build a simple interface or dashboard that consumes the API predictions and cost data, presenting a clear "Sell/Freeze" recommendation based on maximizing predicted net profit for specific batches of produce.
- **Focus Data Collection:** Given the importance of Commodity, Region, and recent price history, ensure robust and timely collection of price data from all economic centers. While weather is important, its lower rank suggests that hyper-accurate weather forecasting might be less critical than accurate price forecasting for this specific model.
- **Implement Data Ingestion:** Develop the placeholder API endpoints (/api/data/weather, /api/data/prices) further to actually store incoming data, enabling future model retraining or online updates.
- **Monitor Model Performance:** Regularly evaluate the model's performance against actual market outcomes (as new data becomes available) and schedule periodic retraining (e.g., monthly or quarterly) to adapt to potential market shifts.

9.4. Limitations

It is important to acknowledge the limitations of the current solution:

- **Static Model:** The current model is trained only on data up to June 2043. It does not automatically adapt to new data submitted via the API (placeholder implementation).
- **Weather Forecasts:** The model uses historical weather data, forward-filled for predictions. Integrating reliable weather forecasts for the prediction horizon would be necessary for true future prediction, adding complexity.

- **External Factors:** The model does not account for unforeseen external events (e.g., sudden disease outbreaks, major policy changes, competitor actions, the "Great Market Eclipse" mentioned in the lore) that could drastically impact prices.
- **Depreciation Assumption:** The model predicts fresh prices; the actual profitability of freezing depends on accurate estimates of depreciation, which are external to this model.

10. Conclusion

This project successfully developed a high-performance time series forecasting system to predict crop prices four weeks ahead, directly addressing the core challenge of the DataCrunch Final Round. By leveraging a LightGBM model with carefully engineered time, lag, and rolling window features, and optimizing hyperparameters using Optuna, the solution achieved an excellent validation RMSE of approximately 20.06.

The system is delivered as a functional FastAPI application within a reproducible Docker container, meeting all specified technical requirements, including the API endpoints for prediction and data submission (placeholders). The solution has been deployed to a public cloud server and is accessible via IP address and domain name. The insights derived from feature importance analysis confirm the significance of commodity, region, and recent price dynamics in driving market prices.

While acknowledging limitations such as the static nature of the current model and the lack of integration with real-time cost data, the developed prediction API provides a powerful tool to inform Magnus Greenvale's "Freezer Gambit" strategy, offering a strong foundation for future enhancements and data-driven decision-making in the Agrovia marketplace.

11. Appendix

11.1. Best Hyperparameters (from Optuna Trial 23)

{

'learning_rate': 0.09661418124610441,

'num_leaves': 57,

'max_depth': 9,

'reg_alpha': 0.02228897246803973,

'reg_lambda': 0.28168407023100606,

'colsample_bytree': 0.8603855854493329,

'subsample': 0.6641597719662065,

'subsample_freq': 3,

'min_child_samples': 36

}

11.2. Full Feature Importance List (Final Tuned Model)

Rank	Feature	Importance
1	Commodity	10202
2	Region	8316
3	Price_lag_42	6130
4	Price_lag_35	6119
5	Price_lag_28	6078
6	Price_roll_mean_7d	5624
7	Price_roll_std_7d	5604
8	Price_roll_std_14d	5331
9	Price_roll_mean_14d	5164
10	Price_roll_std_28d	4466
11	Price_roll_mean_28d	4414
12	Rainfall_mm	4226
13	Humidity_pct	4167
14	Temperature_K	4034
15	CropYieldImpactScore	3847
16	Type	159
17	day	39
18	day_of_week	36
19	day_of_year	31
20	week_of_year	8
21	month	2
22	year	2
23	is_weekend	1

Note: Importance values represent the total number of times a feature was used in splits across all trees in the LightGBM model.